

# Arquitectura de Computadoras

- [Atascos y soluciones](#)
    - [RAW](#)
    - [BTS y BMS](#)
    - [WAW](#)
    - [WAR](#)
    - [STR](#)
  - [Historia de los procesadores RISC](#)
    - [Problema](#)
    - [Estudios](#)
    - [Optimización del uso de registros](#)
      - [Solución por hardware](#)
      - [Solución por software](#)
    - [Conclusiones](#)
  - [Superescalares](#)
    - [Concepto](#)
    - [Paquetes de emisión](#)
    - [Ciclo de instrucción](#)
    - [Políticas de emisión y finalización](#)
    - [Excepciones](#)
- 

## Atascos y soluciones

### RAW (*Read after write*)

- Suceden cuando una instrucción intenta leer un registro que está siendo escrito por una instrucción previa, que aún no terminó.
- Ejemplo:
  - DADD \$t0, \$t5, \$0
  - DADD \$t2, \$t0, \$0
  - La segunda instrucción debe leer \$t0, pero tiene que esperar a que la primera lo termine de escribir.
- Solución: **Forwarding**
  - Hace que los datos de un registro estén disponibles antes de la etapa WB.
    - En una operación aritmética, están disponibles luego de EX.

- Hace que no se lean todos los registros en la etapa ID. En cambio, se leen en la etapa que sean necesarios.
  - En una operación aritmética, los operandos se leen en EX.
  - En un *store*, el registro a guardar se lee en MEM.

## BTS y BMS (*Branch taken stall y branch misprediction stall*)

- Los BTS suceden cuando se toma un salto, pero ya se había captado la siguiente instrucción secuencialmente. Ya que saltó, esa instrucción se captó innecesariamente, perdiendo un ciclo.
- Soluciones: **delay slot y branch target buffer**
- **Delay slot** (salto retardado)
  - Cada instrucción que entre al cauce se ejecutará en su totalidad.
  - En el caso de los saltos, la instrucción siguiente al mismo se ejecutará por cada iteración.
  - Elimina por completo los BTS, aunque no siempre aumenta la eficiencia.
- **Branch target buffer**
  - Intenta predecir si se va a saltar o no, en base a previas iteraciones.
    - Si predijo que iba a saltar, pero en realidad no debía, se producen 2 atascos BMS.
    - Si predijo que no iba a saltar, pero debía, se producen 2 atascos BTS.
      - En ambos casos se pierden 2 ciclos, porque hay un ciclo extra en el que se actualiza la tabla que almacena si se saltó en la previa iteración.

## WAW (*Write after write*)

- Suceden cuando dos instrucciones intentan escribir el mismo registro, pero la segunda podría llegar a terminar antes que la primera. Para que esto no ocurra, suceden los atascos WAW.
- Ejemplo:
  - DDIV \$t4, \$t6, \$t7
  - DADD \$t4, \$s0, \$s2
  - Como la suma requiere de menos ciclos que la división, la instrucción 2 va a escribir \$t4 antes que la instrucción 1. Para evitar esto, se atasca en la etapa EX hasta que termine la etapa DIV de la anterior.

## WAR (*Write after read*)

- Sucede cuando una instrucción intenta escribir un registro, que aún no ha sido leído por una instrucción previa.
- Ejemplo:
  - DMUL \$t1, \$0, \$0
  - DMUL \$t3, \$t1, \$t2
  - DADD \$t2, \$0, \$0
  - La instrucción 2 se va a atascar por RAW en M0 hasta que la instrucción 1 termine la multiplicación, ya que necesita leer \$t1. Mientras sucede este atasco, la instrucción 3 podría seguir, pero estaría escribiendo un registro (\$t2) que la instrucción 2 va a tener que leer una vez que su atasco termine. Para evitar esto, se producen atascos WAR en la etapa ID de la instrucción 3 hasta que la instrucción 2 haya leído \$t2.

## STR (*Structural*)

- Suceden cuando 2 instrucciones intentan acceder a la misma etapa del pipeline en el mismo ciclo. La instrucción 2 se atasca, y solo puede acceder a la etapa una vez la instrucción anterior la libera.
- 

## Historia de los procesadores RISC

### Problema

- Cuando aparecieron los lenguajes de alto nivel, surgió lo que se conoce como **salto semántico**.
  - Es la diferencia entre las instrucciones de alto nivel y las instrucciones de máquina.
    - **Ejemplo:** Un *for* es una sola instrucción en un HLL, pero requiere de muchas más instrucciones a nivel ensamblador.
  - La arquitectura CISC surgió como una solución a este problema, ya que al ser similar a los HLL, reduce el salto semántico.

### Estudios

- Estudio de las operaciones:

|               | <b>Pascal<br/>(Prog.<br/>Científicos)</b> | <b>C (Prog.<br/>estudiantes)</b> | <b>Pascal<br/>(Servicios<br/>sistema)</b> | <b>C (Servicios<br/>sistema)</b> |
|---------------|-------------------------------------------|----------------------------------|-------------------------------------------|----------------------------------|
| <b>Assign</b> | 74                                        | 67                               | 45                                        | 38                               |
| <b>Loop</b>   | 4                                         | 3                                | 5                                         | 3                                |
| <b>Call</b>   | 1                                         | 3                                | 15                                        | 12                               |
| <b>If</b>     | 19                                        | 11                               | 29                                        | 43                               |
| <b>GoTo</b>   | 2                                         | 9                                | -                                         | 3                                |
| <b>Otras</b>  |                                           | 7                                | 6                                         | 1                                |

- Parece que las sentencias de asignación predominan, por lo que sería conveniente optimizar eso. Las llamadas y loops no parecen ser importantes.
  - Así apareció el **CISC**.
- Estudio de cantidad de accesos a memoria y cantidad de instrucciones de máquina:

|               | <b>Aparición dinámica</b> |          | <b>Instrucciones de máquina</b> |          | <b>Referencias a memoria</b> |          |
|---------------|---------------------------|----------|---------------------------------|----------|------------------------------|----------|
|               | <b>Pascal</b>             | <b>C</b> | <b>Pascal</b>                   | <b>C</b> | <b>Pascal</b>                | <b>C</b> |
| <b>Assign</b> | 45                        | 38       | 13                              | 13       | 14                           | 15       |
| <b>Loop</b>   | 5                         | 3        | 42                              | 32       | 33                           | 26       |
| <b>Call</b>   | 15                        | 12       | 31                              | 33       | 44                           | 45       |
| <b>If</b>     | 29                        | 43       | 11                              | 21       | 7                            | 13       |
| <b>GoTo</b>   | -                         | 3        | -                               | -        | -                            | -        |
| <b>Otras</b>  | 6                         | 1        | 3                               | 1        | 2                            | 1        |

- Las instrucciones de loop y llamado son:
  - Las que más accesos a memoria hacen.
  - Las que más instrucciones de máquina generan.
- Por esto deberían optimizarse este tipo de instrucciones, intentando reducir la cantidad de accesos a memoria en las mismas.
- Estudio de operandos

|  | <b>Pascal</b> | <b>C</b> | <b>Promedio</b> |
|--|---------------|----------|-----------------|
|--|---------------|----------|-----------------|

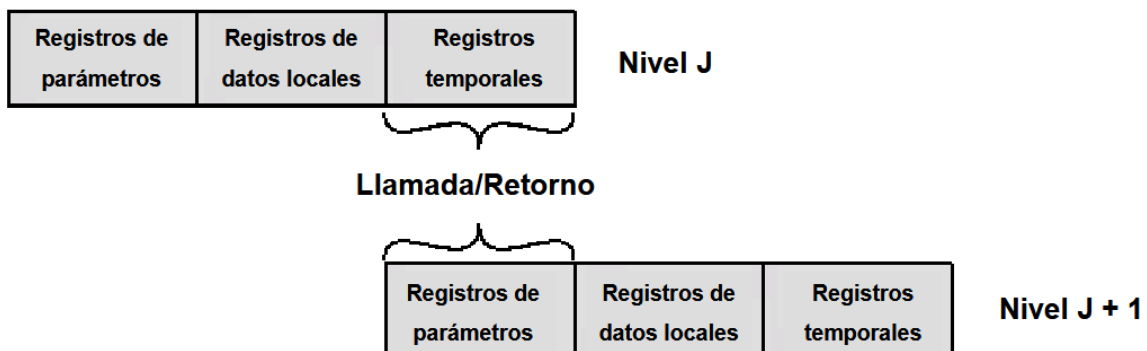
|                            |    |    |    |
|----------------------------|----|----|----|
| <b>Constantes enteras</b>  | 16 | 23 | 20 |
| <b>Variables escalares</b> | 58 | 53 | 55 |
| <b>Vectores y matrices</b> | 26 | 24 | 25 |

- Los operandos más usados son las variables escalares (es decir, las que no son estructuras de datos).
- Estudio de procedimientos
  - El anidamiento de procedimientos es poco común.
  - La gran mayoría de procedimientos usan menos de 6 variables locales, y se les pasan menos de 6 parámetros.

## Optimización del uso de registros

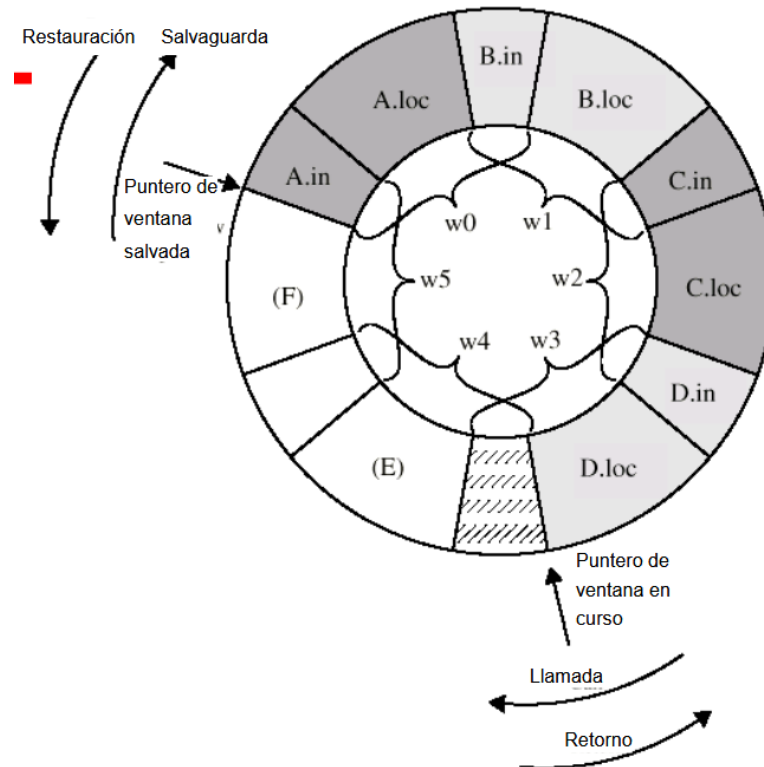
### Solución por hardware: ventana de registros

- La solución por hardware es usar una **ventana de registros**. Debido a los estudios, está focalizada en optimizar el uso de registros en la llamada a procedimientos.
- Los registros de cada procedimiento se dividen en 3: parámetros (lo que recibe), datos locales y temporales:



- Cuando el nivel J llama al nivel J+1, almacena en sus registros temporales los datos que le quiere pasar. Una vez que se está ejecutando en nivel J+1, esos registros dejan de verse como temporales, y ahora se interpretan como parámetros de entrada.
- De esta manera, se evita pasar parámetros usando la pila, lo que reduce la cantidad de accesos a memoria, que era el objetivo.
- Cuando hay muchas llamadas anidadas, pueden acabarse los registros. En ese caso, se deben guardar los registros más antiguos en la pila, para hacer espacio para los nuevos datos que se quieren guardar. Luego, cuando se

retorna, se deben restaurar en los registros aquellos datos que habían sido guardados en la pila.



- Las variables globales, que deben poder ser accedidas desde cualquier procedimiento, se almacenan en un conjunto de registros específicos. No son parte del buffer circular.

### Solución por software: uso del compilador

- El compilador asigna un “registro simbólico” a cada dato que podría llegar a ocupar un registro.
- Como el número de registros simbólicos es infinito, pero el número de registros reales no lo es, el compilador debe:
  - Buscar que varios registros simbólicos compartan el mismo registro físico, siempre y cuando su utilización no se solape en el tiempo
  - Si se queda sin registros físicos, aquellos registros simbólicos que no fueron asignados uno físico se almacenan en memoria.

### Conclusiones

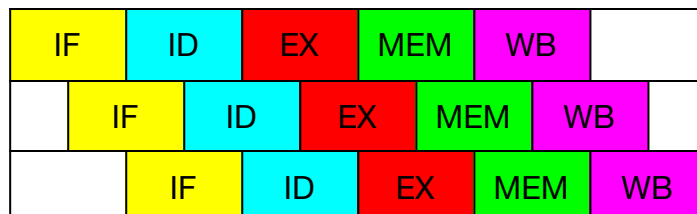
- Un programa se ralentiza por la cantidad de accesos a memoria que se generan en las instrucciones de loop/llamado, por lo que se deberían usar registros en la medida de lo posible.
  - **Resultado**
    - Aumento en la cantidad de registros.

- Ventana de registros.
    - Uso del compilador para optimizar el uso de registros.
  - Finalmente, estas características son las que se corresponden con los procesadores **RISC**.
- 

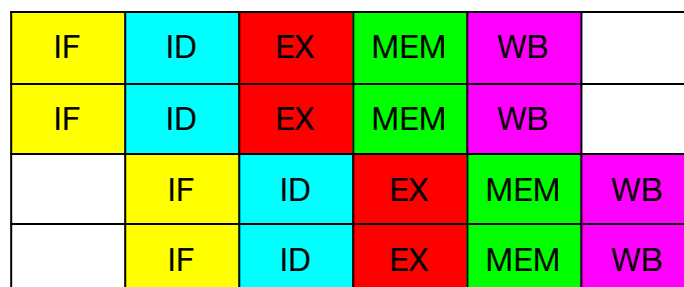
## Superescalares

### Concepto

- Surgieron 2 estrategias para reducir el CPI por debajo de 1: el enfoque supersegmentado y el superescalar.
- **Enfoque supersegmentado**
  - Aprovechando las mejoras de velocidad de los circuitos, se diseña un cauce con un mayor número de etapas de menor duración.
  - Al reducirse el tiempo de cada etapa, el cauce puede funcionar a frecuencias mayores. Se termina cada etapa en menos de un ciclo, por lo que la siguiente instrucción puede empezar antes.
  - El cauce se vería así:



- **Enfoque superescalar**
  - El cauce de estos procesadores puede procesar simultáneamente más de una instrucción por etapa. Esto significa que puede “arrancar” ó emitir varias instrucciones en paralelo.
  - El cauce entonces podría verse así:



- Para lograr esto y evitar atascos estructurales, las unidades funcionales (ALU, unidad de PF, etc deben estar replicadas).

## Paquetes de emisión

- Para que 2 instrucciones puedan ejecutarse en paralelo, deben ser **independientes**.
- El procesador entonces forma **paquetes de emisión**, que son grupos de instrucciones que son independientes entre sí.
- Existen 2 tipos de estrategias para encontrar instrucciones que cumplan dichas condiciones:
  - **Estáticas**: la selección de las instrucciones la realiza el compilador, evitando riesgos de datos y de control (atascos)
  - **Dinámicas**: la selección la realiza el propio procesador. Esta estrategia es la que usan los superescalares

## Ciclo de instrucción

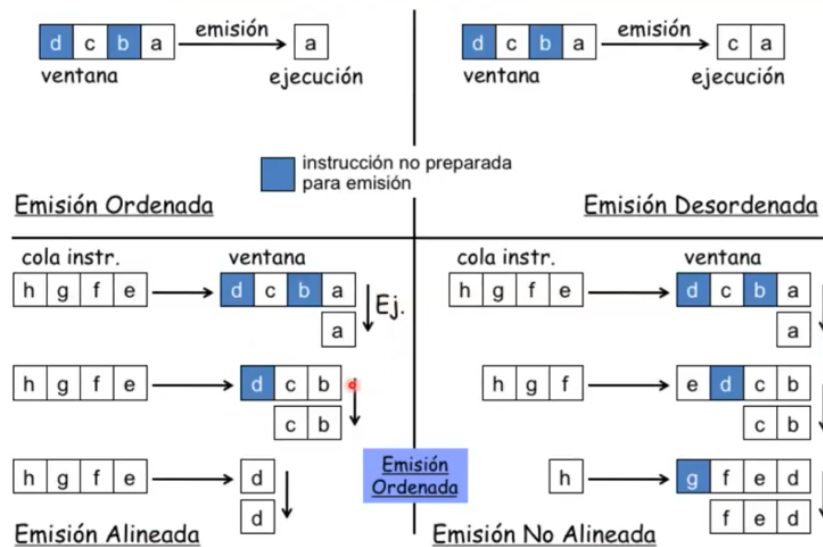
- **Fetch**: se capta la instrucción
- **Decodificación** (*decode*):
  - Se verifica la condición de los saltos.
  - Determinación de la operación (ADD, MUL, etc), localización de operandos y localización del resultado.
- **Ventana de ejecución**
  - Encolado (*dispatch*)
    - Se almacenan las instrucciones ya decodificadas en una **ventana de instrucciones**.
  - Emisión (*issue*)
    - Desde la ventana de instrucciones, se emiten (mandan a ejecutar) aquellas que están listas para ser ejecutadas.
    - Para que una instrucción esté lista, debe tener sus dependencias resueltas, y debe estar libre su unidad de ejecución (puede ser la ALU, PF, etc).
- **Ejecución** (*execute*):
  - En paralelo y en diferentes unidades funcionales.
- **Finalización** (*commit*):
  - El resultado es confirmado en su destino.

## Políticas de emisión y finalización

- Alineación



- Emisión alineada
  - A la hora de hacer el *dispatch*, las instrucciones ingresan a la ventana únicamente cuando esta se libera **totalmente**.
- Emisión no alineada
  - A la hora de hacer el *dispatch*, las instrucciones pueden entrar a la ventana siempre que haya espacio disponible.
- Ordenamiento
  - Emisión ordenada
    - Las instrucciones se emiten desde la ventana en el orden en el que fueron decodificadas. Si la que se decodificó primero se atasca, las siguientes deben esperar.
  - Emisión desordenada
    - Las instrucciones se emiten siempre que sus operandos y unidad funcional ya estén disponibles. Si la que se decodificó primero se atasca, la siguiente se manda igualmente.



- Finalización
  - Finalización ordenada
    - Las instrucciones realizan la escritura de registros en el orden que fueron decodificadas.
  - Finalización desordenada
    - La escritura se realiza a medida que terminan las instrucciones, sin tener en cuenta la secuencialidad del código.
    - **Problema:** atascos WAW y WAR.
    - **Solución:** renombrado de registros.
      - Supongamos un superescalar con 2 unidades de ejecución (ALU y MUL).

- La unidad ALU tendrá un buffer de registros con nombres R1a, R2a, R3a, etc. Lo mismo para la unidad MUL (R1b, R2b, etc).
- Entran estas 2 instrucciones a las unidades de ejecución, en este orden:
  - 1) MUL R3, R5, 2 (Unidad MUL)
  - 2) DADDI R3, R6, 4 (Unidad ALU)
- A la hora de realizar la escritura (que va a suceder en desorden), cada unidad escribe en sus registros locales (R3a y R3b).
  - $R3a := R6 + 4$
  - $R3b := R5 * 2$
- Una vez que ambas instrucciones ya escribieron en sus respectivos registros, se escribe en el R3 “definitivo” en el orden que fueron decodificadas:
  - $R3 := R3b$  (instrucción 1)
  - $R3 := R3a$  (instrucción 2, valor final de R3)
- Este buffer soluciona los riesgos WAW. No importa que la instrucción 2 termine antes que la 1, ya que escribe en R3a, no en R3.
- También solucionan los riesgos WAR, porque las referencias posteriores son a los registros nuevos.

## Excepciones

- Las excepciones son como las interrupciones en la arquitectura CISC.
- Un estado preciso es aquel en el que todas las instrucciones anteriores a la interrupción se ejecutaron, y todas las que la prosiguen se descartaron. Esto asegura la consistencia.
- Cuando sucede una excepción, el comportamiento debería ser idéntico al que tendría un procesador que no es superescalar.
- ¿Qué pasa si hay 2 instrucciones ejecutándose en paralelo y la instrucción 1 es INT 1 (\*)?
  - La instrucción 2 no debería ejecutarse, porque la anterior instrucción fue una excepción.
  - **Para garantizar un estado preciso:**
    - Las instrucciones anteriores a la excepción se completan correctamente
    - La que originó la excepción y las siguientes se abortan.
    - Luego, se comienza por la que originó la excepción.

- (\*) *INT 1 es un ejemplo, no es el nombre real de una excepción.*
- **¿Cómo garantizar un estado preciso si llega una interrupción externa, por ejemplo por hardware?**
  - La unidad de emisión deja de emitir y se cancela la cola.
  - Las instrucciones ya emitidas se completan.
  - Se procesa la interrupción.